

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1023	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 2 – Agile Sprint Planning & User Story Workshop
Weightage:	20%	Total Marks:	25
CO1 Statement:	<i>Apply advanced methodologies and agile project management techniques to manage software projects effectively</i>		
Student Name:	HARINY A P	Reg. No.:	192511334
Date:	09.10.2026	Duration:	60 minutes

Code of Conduct

I, **HARINY A P(192511334)**, certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: **HARINY A P(192511334)**

Chaos Engineering Platform for Distributed Application Resilience

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an Agile Sprint Planning and User Story Workshop by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. Identify the Project Vision and Stakeholders

Define the project's vision, objectives, and identify all key stakeholders involved.

2. Create Epics and User Stories

Break down the requirements into Epics and formulate well-structured User Stories using the format: As a <user>, I want <feature>, so that <benefit>.

3. Define Acceptance Criteria

Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.

4. Prioritize the Product Backlog

Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.

5. Plan the Sprint Backlog

Select high-priority user stories for the first sprint and divide them into actionable development tasks.

6. Estimate Effort Using Story Points

Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.

7. Present the Sprint Goal and Expected Deliverables

Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

1. AGILE SPRINT PLANNING & USER STORY WORKSHOP

Project Vision and Stakeholder Identification

Project Vision

The vision of the Chaos Engineering Platform is to provide a controlled, observable, and safety-oriented environment in which distributed applications can be deliberately exposed to realistic failures before those failures occur unexpectedly in production. The platform will allow development and operations teams to design experiments, inject faults, observe system behaviour, automatically stop risky experiments, and produce evidence-based resilience reports.

The platform transforms resilience testing from an occasional manual activity into a repeatable engineering practice. Instead of checking only whether individual functions work under normal conditions, the system examines whether services continue to operate when dependencies fail, networks become slow, containers terminate, or resources become constrained.

Project Objectives

- Define and execute controlled resilience experiments.
- Simulate realistic service, network, infrastructure, dependency, and resource failures.
- Apply safety limits, monitoring rules, and automatic experiment termination.
- Collect metrics, logs, traces, and experiment events for analysis.
- Integrate resilience experiments into testing and CI/CD workflows.
- Identify weak dependencies and failure-propagation paths.
- Generate evidence-based resilience reports containing observations, failures, and recommendations.

Key Stakeholders

Stakeholder	Role / Interest	Expected Contribution
Development Team	Builds and maintains application services.	Defines service behaviour, dependencies, and test requirements.
DevOps / SRE Team	Responsible for reliability, deployment, and operations.	Designs experiments, monitors health, and evaluates resilience.
QA / Test Engineers	Validates functional and non-functional behaviour.	Creates scenarios, verifies recovery, and records evidence.
Project Manager	Coordinates scope, schedule, and delivery.	Prioritizes backlog and reviews sprint outcomes.
Security / Platform Team	Protects infrastructure and operational boundaries.	Reviews safety controls and access permissions.
Application Owner	Owns business impact and service availability.	Defines critical services and acceptable failure thresholds.
End Users / Customers	Experience application availability and performance.	Provide business expectations for reliability and recovery.

Vision-to-Outcome Traceability

The vision directly supports the stated outcomes: controlled experiments reveal hidden weaknesses, observability provides evidence, safety controls reduce unacceptable impact, and reports convert technical observations into improvement actions.

Stakeholder Dependency

The SRE and platform roles depend on development teams to identify meaningful service boundaries. QA depends on reliable evidence collection to reproduce observations. The project manager uses the prioritized backlog to balance these needs within the available sprint capacity.

2. PRODUCT VISION, SCOPE AND REQUIREMENT DECOMPOSITION

Product Scope and Requirement Decomposition

Product Scope

The proposed product is a software platform for planning, running, observing, and evaluating controlled resilience experiments against distributed applications. The first release focuses on a manageable set of failure types and strong safety controls rather than attempting to reproduce every possible production failure.

In-Scope Capabilities

- Experiment creation with a defined target, failure type, duration, intensity, and rollback behaviour.
- Service failure and container termination experiments.
- Network latency and controlled network degradation experiments.
- Resource-exhaustion simulations such as CPU or memory pressure.
- Dependency failure scenarios for selected downstream services.
- Pre-flight safety validation before an experiment starts.
- Live experiment monitoring and automatic termination when thresholds are exceeded.
- Collection of experiment events, application metrics, logs, and traces.
- Experiment history and comparison of baseline versus chaos results.
- Resilience scoring and evidence-based report generation.
- Basic CI/CD trigger support for repeatable resilience checks.

Out-of-Scope for the Initial Sprint

- Uncontrolled production fault injection.
- Large-scale multi-region disaster simulation.
- Automatic modification of application source code.
- Autonomous decisions to deploy changes directly to production.
- Full enterprise observability replacement.

Major Epics

Epic ID	Epic	Purpose
EP-01	Experiment Management	Create, configure, validate, start, pause, stop, and review experiments.
EP-02	Failure Injection	Apply controlled service, network, resource, container, and dependency failures.
EP-03	Safety & Guardrails	Prevent unsafe experiments and terminate them when risk thresholds are crossed.
EP-04	Observability	Collect and display metrics, logs, traces, and experiment events.
EP-05	Resilience Analysis	Compare baseline and experiment results and identify weak points.
EP-06	Reporting	Generate evidence-based resilience reports and recommendations.
EP-07	CI/CD Integration	Trigger approved resilience experiments from automated delivery workflows.
EP-08	Access & Audit	Control experiment permissions and retain an audit trail.

Functional Requirement Summary

- FR-01: The system shall allow an authorized user to create an experiment configuration.
- FR-02: The system shall validate targets and safety conditions before execution.
- FR-03: The system shall support multiple controlled failure types.
- FR-04: The system shall monitor experiment health and enforce stop conditions.
- FR-05: The system shall record experiment events and relevant observability evidence.
- FR-06: The system shall compare baseline and experiment observations.
- FR-07: The system shall generate a structured resilience report.
- FR-08: The system shall maintain experiment history and audit information.

Requirement Traceability

Each functional requirement is mapped to at least one epic and several user stories. This makes it possible to trace a requirement from the original problem statement through implementation, acceptance testing, and sprint delivery.

System Boundary

The platform controls experiments and records evidence; it does not replace the application, container orchestrator, or observability stack. It acts as an orchestration and analysis layer around those systems.

3. EPICS AND USER STORIES – CORE PLATFORM

The following user stories translate the problem statement into user-centred requirements. Each story follows the required Agile format: “As a <user>, I want <feature>, so that <benefit>.”

ID	Epic	User Story
US-01	Experiment Management	As an SRE engineer, I want to create a chaos experiment with a target service, failure type, duration, and intensity, so that I can test a specific resilience hypothesis.
US-02	Experiment Management	As an SRE engineer, I want to review an experiment before execution, so that I can verify its configuration and safety conditions.
US-03	Failure Injection	As a test engineer, I want to simulate service failure, so that I can determine whether dependent services recover or degrade gracefully.
US-04	Failure Injection	As a test engineer, I want to introduce controlled network delay, so that I can evaluate application behaviour under degraded communication.
US-05	Failure Injection	As a platform engineer, I want to simulate CPU or memory pressure, so that I can evaluate performance and recovery under resource constraints.
US-06	Failure Injection	As an SRE engineer, I want to terminate a selected container, so that I can verify whether orchestration and application redundancy work correctly.

US-07	Failure Injection	As a test engineer, I want to simulate dependency failure, so that I can identify hidden coupling and failure-propagation paths.
US-08	Safety & Guardrails	As an SRE engineer, I want to define experiment stop conditions, so that unsafe behaviour automatically terminates the experiment.
US-09	Safety & Guardrails	As a project administrator, I want experiments to require authorization and validation, so that only approved resilience tests can run.
US-10	Safety & Guardrails	As an operations engineer, I want an emergency stop mechanism, so that I can immediately halt an experiment when unexpected risk appears.

User Story Quality Notes

- Each story has one primary actor and one clearly identifiable capability.
- The benefit explains why the capability matters to resilience engineering.
- Stories are independently testable where practical and can be estimated using relative story points.
- High-risk infrastructure actions are separated from analysis and reporting features to make sprint planning manageable.
- Safety-related stories receive high priority because the platform itself must not become a source of unacceptable operational impact.

INVEST Check

The stories are designed to be Independent where practical, Negotiable during refinement, Valuable to a named stakeholder, Estimable through story points, Small enough for sprint planning, and Testable through acceptance criteria.

Story Dependencies

US-01 and US-02 form the configuration foundation. US-08 and US-10 provide safety before disruptive actions are expanded. US-13 and US-14 provide the evidence needed to make later analytical stories meaningful.

4. EPICS AND USER STORIES – OBSERVABILITY, ANALYSIS & REPORTING

ID	Epic	User Story
US-11	Observability	As an SRE engineer, I want to view live experiment status and health indicators, so that I can observe system impact while the experiment is running.
US-12	Observability	As a QA engineer, I want experiment events to be timestamped, so that I can reconstruct what happened during a failure scenario.
US-13	Observability	As an operations engineer, I want metrics, logs, and traces associated with an experiment, so that I can connect injected failures with observed system behaviour.

US-14	Resilience Analysis	As an SRE engineer, I want to compare baseline and experiment results, so that I can quantify the effect of a failure.
US-15	Resilience Analysis	As a reliability engineer, I want the platform to identify degraded services and dependency paths, so that I can locate resilience weaknesses.
US-16	Resilience Analysis	As an application owner, I want a resilience score based on measurable indicators, so that I can track improvement over time.
US-17	Reporting	As a project manager, I want an experiment report containing configuration, observations, failures, and recovery evidence, so that I can review the result without manually collecting data.
US-18	Reporting	As an SRE engineer, I want recommendations linked to observed weaknesses, so that the team can plan corrective actions.
US-19	CI/CD Integration	As a DevOps engineer, I want approved resilience experiments to be triggered from CI/CD, so that resilience checks become repeatable.
US-20	Access & Audit	As an administrator, I want an audit trail of experiment creation, approval, execution, and termination, so that actions remain traceable.

Non-Functional Expectations

- Safety: Experiments must operate within explicitly configured boundaries and support immediate termination.
- Reliability: Experiment records should not be lost when the target application experiences a failure.
- Observability: Every experiment should provide enough evidence to explain what was injected and what changed.
- Usability: An authorized user should be able to understand experiment status without reading raw infrastructure logs.
- Security: Only authorized users should be able to create, approve, or execute disruptive experiments.
- Auditability: Experiment actions and outcomes should be traceable by time, user, target, and experiment identifier.
- Scalability: The architecture should allow additional failure types and target environments to be added without redesigning the entire platform.

Key Resilience Indicators

- Service availability during the experiment.
- Latency increase compared with baseline.
- Error-rate increase compared with baseline.
- Time to detect the injected failure.
- Time to recover after failure removal.
- Number of affected dependent services.
- Percentage of experiment duration within acceptable service-level thresholds.

Evidence Quality

Telemetry should be associated with an experiment identifier and timestamps. This prevents observations from different experiments from being mixed and makes later comparison more reliable.

Operational Usefulness

The analysis should distinguish an expected effect from an unacceptable failure. A temporary latency increase within the defined threshold may be an expected observation, while a sustained availability breach should be treated as a resilience finding.

5. ACCEPTANCE CRITERIA – EXPERIMENT AND SAFETY STORIES

Acceptance criteria are written using the Given–When–Then structure so that each selected story can be objectively verified.

US-01 – Create Experiment

Given an authorized SRE user is logged in, when the user enters a valid target, failure type, duration, and intensity, then the platform shall create the experiment in a draft or ready state.

Given required experiment fields are missing, when the user attempts to save the experiment, then the platform shall identify the missing information and prevent invalid execution.

US-02 – Review Experiment

Given an experiment is configured, when the user opens the review screen, then the platform shall display the target, failure parameters, duration, safety limits, and expected impact.

Given a required safety condition is not satisfied, when the user attempts to start the experiment, then execution shall be blocked.

US-03 – Service Failure

Given a permitted target service is selected, when the approved experiment starts, then the configured service-failure action shall be applied for the specified duration.

Given the failure is active, when dependent services are monitored, then the platform shall record observable changes such as errors, latency, or availability.

US-04 – Network Delay

Given a valid network-delay experiment is approved, when execution begins, then the configured delay shall be applied only to the selected target path.

Given the delay causes a configured risk threshold to be exceeded, when the monitoring rule detects the threshold, then the experiment shall be terminated automatically.

US-05 – Resource Exhaustion

Given a permitted target has resource limits configured, when the experiment starts, then controlled resource pressure shall be applied within the defined limit.

Given resource pressure exceeds the configured safety boundary, when the guardrail detects the breach, then the experiment shall stop and record the termination reason.

US-06 – Container Termination

Given a redundant target service is approved for testing, when the experiment begins, then the selected container shall be terminated according to the experiment configuration.

Given the orchestration system starts a replacement, when recovery occurs, then the platform shall record the recovery event and timing.

US-07 – Dependency Failure

Given a downstream dependency is selected, when the experiment starts, then the platform shall apply the configured dependency failure without affecting unrelated targets.

Given dependent services degrade, when the experiment completes, then affected services and observed propagation shall be included in the evidence.

Acceptance Test Principle

Acceptance criteria focus on observable behaviour rather than implementation details. The same criterion can therefore be validated whether the underlying platform uses one technical mechanism or another.

Safety Before Scale

The first safety stories are intentionally conservative. The platform should prove that it can stop an experiment safely before it attempts broader failure combinations or higher-impact scenarios.

6. ACCEPTANCE CRITERIA – OBSERVABILITY, ANALYSIS & REPORTING

US-08 – Automatic Safety Termination

Given an experiment has defined stop conditions, when any configured threshold is exceeded, then the platform shall stop the experiment automatically.

Given an automatic stop occurs, when the experiment is closed, then the termination condition, timestamp, and observed threshold shall be recorded.

US-10 – Emergency Stop

Given an authorized operations user has access to the running experiment, when the emergency-stop action is invoked, then the experiment shall move to a terminating state immediately.

Given an emergency stop is executed, when the result is recorded, then the platform shall preserve the reason and actor responsible for the action.

US-11 – Live Monitoring

Given an experiment is running, when the user opens the monitoring view, then current experiment state and configured health indicators shall be visible.

Given the target becomes unhealthy, when the monitoring system receives the evidence, then the status shall reflect the degraded condition.

US-13 – Evidence Collection

Given an experiment is active, when application and platform telemetry is available, then relevant metrics, logs, traces, and experiment events shall be associated with the experiment.

Given an experiment ends, when the evidence is stored, then it shall remain accessible for later analysis.

US-14 – Baseline Comparison

Given baseline measurements and experiment measurements exist, when the user requests comparison, then the platform shall show meaningful differences such as latency, error rate, and availability.

Given a measurable degradation is found, when the analysis is generated, then the affected indicator shall be highlighted.

US-15 – Weakness Identification

Given experiment evidence contains service or dependency degradation, when the analysis is executed, then the platform shall identify affected components and relevant propagation paths.

Given no significant degradation is detected, when the analysis completes, then the result shall indicate that no tested weakness was observed under the selected conditions.

US-17 – Resilience Report

Given an experiment has completed, when the user requests a report, then the report shall include configuration, hypothesis, execution timeline, observations, failures, recovery evidence, and conclusions.

Given the report is generated, when the user reviews it, then the report shall clearly distinguish observed evidence from recommendations.

Analysis Evidence

A resilience conclusion should be supported by at least one measurable observation such as availability, latency, error rate, recovery time, or affected dependency count. This keeps the report evidence-based rather than opinion-based.

Recovery Verification

Recovery should be checked after fault removal. Detecting a failure is not sufficient; the platform should also capture whether the affected service returns to an acceptable operating state.

7. PRODUCT BACKLOG PRIORITIZATION

The product backlog is prioritized using business value, resilience impact, dependency order, technical risk, and safety importance. MoSCoW thinking is used alongside ranking: Must Have items form the minimum usable resilience platform, while Should Have and Could Have items extend analysis and automation.

Rank	Story	Backlog Item	MoSCoW	Value	SP	Priority Rationale
1	US-08	Automatic safety termination	Must	Critical	5	Required before disruptive experiments can be trusted.
2	US-10	Emergency stop	Must	Critical	3	Provides immediate human control over active experiments.
3	US-01	Create experiment	Must	High	5	Foundation for defining every experiment.
4	US-02	Review and validate experiment	Must	High	3	Prevents invalid or unsafe configurations.
5	US-11	Live monitoring	Must	High	5	Necessary to observe system impact during execution.
6	US-03	Service failure injection	Must	High	5	Core chaos capability for resilience testing.
7	US-04	Network delay injection	Must	High	5	Tests a common distributed-system failure mode.
8	US-13	Collect observability evidence	Must	High	8	Without evidence, experiment results cannot be meaningfully evaluated.
9	US-14	Baseline comparison	Should	High	5	Converts raw telemetry into measurable impact.
10	US-05	Resource exhaustion	Should	Medium	5	Adds infrastructure-pressure testing.
11	US-06	Container	Should	Medium	3	Tests redundancy and

		termination				restart behaviour.
12	US-07	Dependency failure	Should	High	5	Useful for identifying propagation paths.
13	US-15	Weakness identification	Should	High	8	Supports resilience analysis and corrective planning.
14	US-17	Resilience report	Should	Medium	5	Provides structured evidence for stakeholders.
15	US-16	Resilience score	Could	Medium	5	Enables longer-term tracking.
16	US-18	Recommendations	Could	Medium	5	Helps convert findings into improvement actions.
17	US-19	CI/CD trigger	Could	Medium	8	Enables repeatable automation after the core platform is stable.
18	US-20	Audit trail	Could	Medium	3	Improves traceability and governance.

Prioritization Rationale

Safety controls are ranked first because a chaos engineering platform must establish its own operational safety before introducing failures. Experiment configuration and validation follow because they provide the basic workflow. Monitoring and evidence collection are prioritized next because a failure without observation cannot provide useful resilience evidence. Advanced analytics, reporting, and CI/CD integration are scheduled after the execution and safety foundation is stable.

Priority Rules

When two stories have similar business value, the story that removes a safety or dependency risk receives the higher priority. This prevents the team from delivering attractive reporting features before the execution foundation is trustworthy.

Backlog Refinement

Before sprint commitment, stories should be reviewed for unclear targets, missing acceptance criteria, technical dependencies, and oversized scope. Any story that cannot be understood and tested should remain in refinement.

8. SPRINT BACKLOG – SPRINT 1

Sprint Duration: 2 weeks

Sprint Theme: Build a safe minimum viable chaos-experiment workflow with controlled failure injection, monitoring, and evidence capture.

Story	Sprint Work Item	Development Tasks	SP
US-01	Create experiment configuration	Design experiment form; validate fields; store draft; generate experiment ID.	5

US-02	Review and validate experiment	Implement pre-flight checks; target validation; safety checklist; approval state.	3
US-08	Automatic safety termination	Define thresholds; monitor conditions; implement stop logic; record termination event.	5
US-10	Emergency stop	Add authorized stop action; terminate active experiment; record user and reason.	3
US-03	Service failure injection	Implement controlled service-failure action; duration control; restore target.	5
US-11	Live monitoring	Show experiment status, health indicators, current phase, and alerts.	5
US-13	Evidence collection	Capture experiment events and selected telemetry references; link evidence to experiment ID.	8
US-14	Basic baseline comparison	Store baseline values and show simple comparison after experiment completion.	5

Task Breakdown by Workstream

- Planning & design: define experiment lifecycle, states, safety rules, and data model.
- Backend: implement experiment creation, validation, execution states, and termination logic.
- Failure engine: implement the first controlled service-failure scenario behind an approved interface.
- Monitoring: expose live experiment state and selected health indicators.
- Evidence: record timestamps, actions, termination reasons, and telemetry references.
- Frontend: create experiment setup, review, monitoring, and result screens.
- Testing: perform unit, integration, safety-boundary, and recovery tests.
- Documentation: prepare experiment instructions and Sprint 1 evidence.

Definition of Done

- Code is reviewed and integrated into the sprint branch.
- Invalid configurations are rejected before execution.
- Safety thresholds are tested using controlled scenarios.
- Emergency stop is functional and auditable.
- The first failure experiment can be executed only after validation.
- Experiment evidence is associated with a unique experiment identifier.
- Basic monitoring and baseline comparison are demonstrable.
- Known limitations and residual risks are documented.

Sprint Dependencies

The sprint sequence starts with configuration and validation, followed by safety and controlled execution, then monitoring and evidence. This order reduces the risk of building a failure injector without the controls needed to operate it safely.

Sprint Tracking

Daily Scrum discussion should focus on progress toward the sprint goal, blockers involving infrastructure access, and any discovery that changes the effort or risk of the selected stories.

9. STORY POINT ESTIMATION AND PLANNING POKER

The team uses Planning Poker with the modified Fibonacci sequence 1, 2, 3, 5, 8, 13. Story points represent relative effort, technical complexity, uncertainty, and integration risk rather than hours.

Story	Feature	Story Points	Estimation Justification
US-01	Create experiment	5	Several UI/data validations and a new experiment lifecycle, but limited infrastructure complexity.
US-02	Review and validation	3	Mostly validation and workflow logic; depends on US-01.
US-03	Service failure	5	Requires safe execution, target control, timing, and recovery handling.
US-04	Network delay	5	Requires controlled network manipulation and strict targeting.
US-05	Resource exhaustion	5	Requires bounded resource pressure and monitoring.
US-06	Container termination	3	Focused action with recovery observation, assuming orchestration support exists.
US-07	Dependency failure	5	Requires dependency targeting and analysis of downstream effects.
US-08	Safety termination	5	Cross-cutting guardrail with monitoring and termination behaviour.
US-10	Emergency stop	3	Small feature but operationally critical and must be reliable.
US-11	Live monitoring	5	Requires live state updates and health visualization.
US-13	Evidence collection	8	Integration with multiple telemetry sources introduces uncertainty.
US-14	Baseline comparison	5	Requires consistent baseline storage and comparison logic.
US-15	Weakness identification	8	Combines evidence interpretation and dependency-path analysis.
US-17	Resilience report	5	Requires aggregation and presentation of experiment evidence.

Why Fibonacci Story Points?

The gaps between larger Fibonacci values represent increasing uncertainty. A small story such as an emergency stop has a relatively clear implementation path, while evidence collection and weakness identification involve several integrations and unknowns. If a story reaches 13 points, the team should normally split it into smaller stories before placing it into a sprint.

Sprint Capacity Check

Sprint 1 contains 39 story points. The allocation is intentionally weighted toward safety, execution, monitoring, and evidence because these features form the minimum safe workflow. If actual team capacity is lower, the first items to defer are basic baseline comparison and part of evidence integration, while the safety controls remain mandatory.\

Estimation Factors

The team considers implementation effort, integration complexity, uncertainty, testing difficulty, and operational risk when assigning points. A technically small change can still receive a higher estimate when its failure impact makes testing more demanding.

Re-estimation Rule

Story points should be revisited only when new information changes the relative size of a story. They should not be converted into hours simply to make the estimate appear more precise.

10. SPRINT GOAL, DELIVERABLES AND ACCEPTANCE OF SPRINT

Sprint Goal

Build and demonstrate a safe minimum viable chaos engineering workflow that allows an authorized user to configure and validate an experiment, execute one controlled service failure, observe the target, automatically stop the experiment when a safety threshold is breached, manually stop it when necessary, and preserve evidence for later analysis.

Expected Sprint Deliverables

- Experiment configuration and unique experiment identifier.
- Pre-flight validation and safety checklist.
- Approved experiment lifecycle: Draft → Validated → Running → Stopped/Completed.
- Controlled service-failure experiment.
- Automatic safety termination mechanism.
- Authorized emergency-stop mechanism.
- Live experiment monitoring view.
- Experiment event and telemetry evidence record.
- Basic baseline-versus-experiment comparison.
- Test cases covering normal execution, invalid configuration, threshold breach, and emergency stop.
- Sprint demonstration and concise technical documentation.

Sprint Acceptance Checklist

No.	Acceptance Check	Result
1	Experiment can be created with mandatory fields.	Pass/Fail
2	Invalid target or unsafe configuration is blocked.	Pass/Fail
3	Authorized user can start an approved experiment.	Pass/Fail
4	Controlled service failure affects only the selected target.	Pass/Fail
5	Safety threshold causes automatic termination.	Pass/Fail
6	Emergency stop terminates a running experiment.	Pass/Fail
7	Experiment status is visible while running.	Pass/Fail
8	Events and termination reasons are retained.	Pass/Fail

9	Baseline and experiment observations can be compared.	Pass/Fail
10	A sprint demonstration can reproduce the complete workflow.	Pass/Fail

Sprint Review Focus

The Sprint Review should demonstrate the complete workflow rather than isolated screens: configure an experiment, validate it, execute the controlled failure, observe the target, trigger or reach a safety boundary, terminate the experiment, and review the evidence. This directly demonstrates that the platform addresses the central problem statement.

Sprint Retrospective Questions

- Were safety rules clear enough before execution?
- Did monitoring provide sufficient evidence to explain the experiment?
- Which failure-injection step introduced the most technical uncertainty?
- Did the team estimate infrastructure integration effort accurately?
- What should be changed before adding additional failure types?

Sprint Review Evidence

The demonstration should include a before-and-after view: baseline health, injected failure, observed impact, termination behaviour, and recovery evidence. This provides stakeholders with a concrete view of the sprint goal.

Sprint Success Measure

Sprint 1 is successful when the team can safely reproduce the complete experiment lifecycle and explain the observed result using stored evidence, not merely when individual interface screens are completed.

11. AGILE ROADMAP, RISKS AND EXPECTED OUTCOMES

Future Sprint Roadmap

Sprint	Focus	Major Outcomes
Sprint 1	Safe MVP	Experiment lifecycle, service failure, safety termination, emergency stop, monitoring, evidence.
Sprint 2	Failure Expansion	Network delay, resource exhaustion, container termination, dependency failure.
Sprint 3	Resilience Analysis	Baseline comparison, failure propagation analysis, resilience indicators and scoring.
Sprint 4	Reporting & Automation	Evidence-based reports, recommendations, audit trail, CI/CD triggers.
Sprint 5	Hardening	Security review, performance testing, usability improvements, recovery validation and final documentation.

Project Risks and Mitigation

Risk	Impact	Mitigation
Unsafe experiment affects a critical service	Very High	Use isolated test targets, pre-flight validation, strict thresholds, authorization, and emergency stop.
Failure injection is difficult to reproduce	High	Version experiment configurations and use repeatable parameters.
Insufficient observability	High	Define required evidence before execution and associate telemetry with experiment IDs.
Experiments cause cascading failures	Very High	Limit blast radius, target specific services, and enforce automatic termination.
CI/CD trigger executes unexpectedly	High	Require approved experiment profiles and explicit pipeline permissions.
Large scope delays delivery	Medium	Use prioritized epics and keep advanced analytics outside the MVP.
False conclusions from noisy metrics	Medium	Use baseline measurements, repeated experiments, and clear acceptance thresholds.

Expected Outcomes

- Hidden weaknesses in distributed systems can be identified before they become production incidents.
- Application fault tolerance can be improved using evidence rather than assumptions.
- Unexpected production failures can be reduced by testing realistic failure conditions.
- Testing can be strengthened beyond conventional functional scenarios.
- Incident preparedness and recovery confidence can be improved.
- Measurable resilience indicators can support continuous improvement.
- The platform can provide a repeatable foundation for controlled chaos engineering within an organization's testing lifecycle.

Conclusion

The Agile plan converts the Chaos Engineering Platform problem statement into a structured, deliverable-oriented backlog. The first sprint deliberately concentrates on safety, controlled execution, monitoring, and evidence because these capabilities are prerequisites for trustworthy chaos experiments. Subsequent sprints can expand failure types, analytical capabilities, reporting, and CI/CD integration while preserving the same guardrail-first approach.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria and Mark

Criteria	Mark
Project Vision & Stakeholder Identification (30%)	/5
Epics & User Stories (25%)	/5
Acceptance Criteria (20%)	/5
Product Backlog Prioritization & Sprint Planning (15%)	/5
Story Point Estimation, Sprint Goal & Presentation (10%)	/5
TOTAL	/25